Nios® V/g Processor OCM Memory Test Design

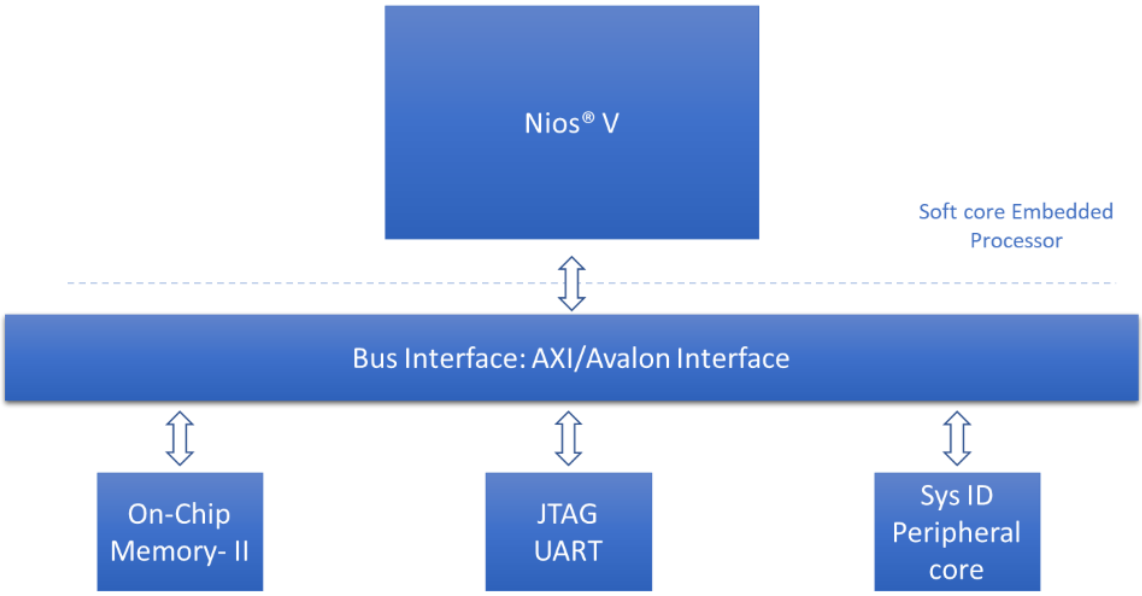
Agilex™ 5 FPGA

Contents

1. Theory of Operation

Nios® V/g Processor-based OCM memory test example design on the Agilex™ 5 FPGA E-Series 065B Premium Development Kit (ES1) DK-A5E065BB32AES1.

Figure 1-1. Block Diagram



1.1. IP Cores

The following IPs are used in this design.

- Nios V/g soft processor core
- On Chip RAM-II
- JTAG UART
- System ID

2. Directory Structure

The directory structure is explained below:

Root directory: `niosv_g/niosv_g_ocm_mem_test`

Directories	Content
docs	Document capturing the details of the example design
img	Block diagram for the example designs
ready_to_test	Prebuilt binaries (FPGA `.sof` file and Application `.elf` file) which can be programmed on the board and tested
sources - hw - scripts - sw/app	Files needed to create the design - Necessary hardware files (`.sv, .v, .ip`) of the design - This folder consists of scripts to build the design - This folder contains software application files

3. Quick Start Guide

You may try the example on the target development kit using the prebuilt binaries. The respective FPGA `sof` and Application `elf` files are found in `ready_to_test` folder.
Refer to Chapter 6 Programming the Design for the steps to program these files.

4. Generating the Design

The implementation of Nios® V processor system requires a hardware design developed using the Quartus® Prime software. After that, it is required to develop the software design that complements the processor system.

4.1. Building Hardware Design

Begin designing the system hardware by instantiating the Nios® V processor and its peripherals into Platform Designer. After configuring the system assignments and constraints, complete the hardware design by performing a successful compilation.

We will be using `sources/scripts/build_sof.py` to develop the hardware design.

This script will create a new platform design, add the IPs into the design, makes the connections, compiles the design and generates the .sof file.

Steps to execute the script are as follows:

1. Invoke the Nios V Command Shell in the terminal
2. Run the following command in the terminal from top level project directory: `> quartus_py ./scripts/build_sof.py`
3. The Quartus tool will compile the design and generate the output files into `<Root Directory>/sources/hw/output_files`

4.2. Building Software Design

After the processor system is ready, you may begin building the software design. It consists of the following steps:

1. Create a board support package (BSP) project.
2. Create a Nios® V processor application project with source code.
3. Build the application.
4. Convert the application .elf file into .hex file for memory initialization.

To create software application, run the following commands in the terminal:

```
> niosv-bsp -c --quartus-project=hw/top.qpf --qsys=hw/qsys_top.qsys --
type=hal sw/bsp/settings.bsp

> niosv-app --bsp-dir=sw/bsp --app-dir=sw/app --srcs=sw/app/main.c

> niosv-shell

> cmake -S ./sw/app -B sw/app/build -G "Unix Makefiles"

> make -C sw/app/build

> elf2hex sw/app/build/app.elf -b 0x0 -w 32 -e 0x9ffff
sw/app/build/onchip_mem.hex -r4
```

Note: It is recommended to clean the `app/build` project before regenerating `elf` (cmake and make).

5. Simulating the Design

You may carry on simulation for this design to evaluate its intended functions. During simulation, the action of downloading .elf file is unable to be simulated. Thus, the processor memory is instead initialized with the application .hex file.

Use the following commands to run the simulation:

```
# Generate Testbench from Platform Designer.

# Generate -> Generate Testbench System

> cp ./sw/app/build/onchip_mem.hex ./qsys_top_tb/qsys_top_tb/sim/mentor

> cd hw/qsys_top_tb/qsys_top_tb/sim/mentor/

> vsim &

> source msim_setup.tcl

> ld_debug

> run -all
```

6. Programming the Design

Use the Quartus® Prime Programmer tool and the Ashling* RiscFree* IDE to program the Nios® V processor-based system into the FPGA and to run your application.

Program the generated `sof`, then download the `elf` file on the board according to the command below.

```
> quartus_pgm --cable=1 -m jtag -o 'p;ready_to_test/top.sof'

#----- Download the elf file on the board ----#

> niosv-download -g ready_to_test/app.elf -c 1

#----- Verify the output on the terminal by using the following command in the terminal -----#

> juart-terminal -d 1 -c 1 -i 0
```

Note: Reduce the JTAG clock frequency to 6MHz before programming the application `.elf` file on the board using the command: `jtagconfig --setparam 1 JtagClock 6M`

7. Expected Results

The following is the output as observed on the JTAG UART terminal. The output is analogous to the logic from the application code. Users should be able to observe the same output on their terminal/setup.

Figure 1-2. Expected Result

```
Hello World from NIOSV/g core !
Application will execute Memory Test
Starting Memory test
value at memory location 0x0 with offset 0 is 0x4600006f
After write: value at memory location 0x0 with offset 0 is 0xa5a5a5a5
Memory write test PASSED
value at memory location 0x0 with offset 4 is 0x0
After write: value at memory location 0x0 with offset 4 is 0xa5a5a5a5
Memory write test PASSED
value at memory location 0x0 with offset 8 is 0x0
After write: value at memory location 0x0 with offset 8 is 0xa5a5a5a5
Memory write test PASSED
value at memory location 0x0 with offset 12 is 0x0
After write: value at memory location 0x0 with offset 12 is 0xa5a5a5a5
Memory write test PASSED
value at memory location 0x0 with offset 16 is 0x0
After write: value at memory location 0x0 with offset 16 is 0xa5a5a5a5
Memory write test PASSED
value at memory location 0x0 with offset 20 is 0x0
After write: value at memory location 0x0 with offset 20 is 0xa5a5a5a5
Memory write test PASSED
value at memory location 0x0 with offset 24 is 0x0
After write: value at memory location 0x0 with offset 24 is 0xa5a5a5a5
Memory write test PASSED
value at memory location 0x0 with offset 28 is 0x0
After write: value at memory location 0x0 with offset 28 is 0xa5a5a5a5
Memory write test PASSED
Memory Test Complete
Print the value of System ID
System ID from Peripheral core is 0xA5
```